ClinIQ — Autonomous Clinical Intelligence Platform

The Problem

Healthcare organizations are drowning in data and starving for insight at the same time.

A nurse manager wants to know the 30-day readmission rate for heart failure patients. A quality officer needs the percentage of diabetic patients with an A1c above 9% last quarter. A finance director wants to see ER visit volume trends month over month. None of these people can write SQL. All of them are blocked on a data analyst who has fifteen other requests queued up.

This is called the analyst bottleneck. It exists in every hospital, every health system, every health tech company. It is expensive in two ways. In clinical settings, delayed data means delayed decisions — which can affect patient outcomes. In operational settings, it slows every initiative that depends on data, which is increasingly everything.

The root cause is a translation problem. The people who have clinical questions don't speak SQL. The databases don't speak English. And the pipelines feeding those databases were designed for batch reporting, not real-time operational questions.

The Why — Why Does This Matter to Build?

The problem is real and unsolved at scale. EHR vendors like Epic have reporting modules but they are rigid, require IT configuration months in advance, and cannot handle ad-hoc questions. BI tools like Tableau require trained analysts to build views first. LLM-powered SQL tools exist but are cloud-locked, expensive, and not deployable in privacy-sensitive environments. A locally deployable, privacy-preserving alternative has genuine value.

Healthcare is the highest-stakes vertical for AI correctness. When an agent answers a clinical operations question incorrectly, decisions that affect patients can be made on bad data. That makes the engineering of correctness — validation, self-correction, auditability, explainability — not optional but essential. Building this in a healthcare context forces rigor that a sales analytics agent does not. That rigor is visible in the architecture and impressive in an interview.

The technical surface is exactly what the 2026 market is hiring for. Streaming pipelines, lakehouse storage formats, RAG, agentic frameworks, MCP — these are the specific signals that appear in AI engineer and data engineer job descriptions right now. This project hits all of them through one coherent system rather than five disconnected toy projects.

What ClinIQ Does — The End Product

ClinIQ is a locally deployable clinical intelligence platform. A non-technical clinical user opens a web interface, types a question in plain English, and receives a direct answer in plain English, an interactive chart, the SQL that was run (for transparency and auditability), and a suggested follow-up question.

The system can answer two fundamentally different kinds of questions:

Structured questions — things answerable from the patient database. "How many diabetic patients were admitted last month?" "What is the average length of an inpatient stay?" "Which conditions are most common in patients over 65?" These go through a Text-to-SQL path: the agent generates SQL, runs it against the clinical database, validates the result, and synthesizes an explanation.

Unstructured questions — things answerable from clinical documents and guidelines. "What does the ADA recommend for A1c targets in elderly patients?" "What are the readmission risk criteria for heart failure?" These go through a RAG path: the agent searches a vector store of clinical guidelines using semantic similarity, retrieves the most relevant passages, and synthesizes a cited answer.

The agent knows which path to use based on what the question is asking. It routes automatically. And when SQL fails, it corrects itself — the error message goes back to the generator with context and it tries again.

All inference runs locally. No patient data, even synthetic, ever leaves the machine. That is not just a cost decision — it is the architecture that real clinical deployments require.

The Full Phased Build Plan

Phase 0 — Project Setup and Environment

Goal: Repository structure, dependency installation, Docker Compose skeleton, and all services defined before any logic is written.

Directory structure:

cliniq/

├— data/

```
│   ├── raw/            ← Synthea CSVs
│   └── parquet/        ← Delta/Parquet tables
├── dbt_project/
│   ├── models/
│   │   ├── staging/
│   │   ├── marts/
│   │   └── metrics/
│   └── seeds/          ← reference CSV files (drug classes, LOINC ranges)
├── airflow_dags/
├── great_expectations/
├── agent/
│   ├── nodes/
│   ├── tools/
│   ├── graph.py
│   └── prompts.py
├── vector_store/
├── mcp_servers/
│   ├── database_server.py
│   └── vector_server.py
├── api/
├── frontend/
├── mlflow_tracking/
├── docker/
│   └── docker-compose.yml
└── docs/
```

├── schema_registry.json

└── architecture.md

Docker Compose services defined now, started progressively:

- postgres — PostgreSQL 15 with pgvector extension
- ollama — local LLM inference server
- airflow — Airflow standalone
- mlflow — MLflow tracking server
- api — FastAPI agent backend
- streamlit — Streamlit frontend

You don't start all of these in Phase 0. You define the full compose file now so the intended architecture is explicit, then start services as each phase needs them. This is a professional habit — it forces you to think about the full system before you get buried in individual components.

Python dependencies installed in Phase 0: dbt-core, dbt-duckdb, great-expectations, apache-airflow, deltalake, pyarrow, duckdb, psycopg2-binary, pgvector, sentence-transformers, langchain, langchain-ollama, langgraph, mcp (Anthropic's Python SDK), fastapi, uvicorn, streamlit, plotly, mlflow.

Phase 1A — Data Generation and Raw Storage (Synthea + Delta Lake + Parquet)

Goal: Generate a realistic synthetic clinical dataset and store it in a modern lakehouse format rather than flat CSVs.

Synthea — why this dataset:

Synthea is an open-source synthetic patient generator built by MITRE Corporation. It produces statistically calibrated fake patient records using actual clinical coding standards — ICD-10 for diagnoses, LOINC for lab results, RxNorm for medications. The patients have complete longitudinal histories — diagnoses that precede related medications, lab trends that reflect disease progression, multiple encounters over time. This structural complexity is what makes the project feel like real work rather than a tutorial.

You run it with a fixed random seed so your benchmark answers are stable across regenerations: ./run_synthea -p 5000 -s 42 Massachusetts. Five thousand patients gives

you enough data to produce meaningful aggregate statistics without slowing down local queries.

Spend time with the raw CSVs before writing any code. Understand that conditions.STOP is null for ongoing chronic conditions. That observations.VALUE stores both numeric lab values and text codes in the same string column. That encounters.ENCOUNTERCLASS is a controlled vocabulary (ambulatory, emergency, inpatient, wellness, urgentcare, outpatient). These quirks inform every modeling decision downstream.

Delta Lake and Parquet — why not just load CSVs:

The first thing you do with the Synthea CSVs is convert them to Parquet files in Delta format. This is the foundational architectural decision that separates this project from tutorials.

Parquet is a columnar binary storage format. A CSV stores every row sequentially — to answer "what is the average A1c across all patients," it reads every column of every row. Parquet stores each column together — the same query reads only the value column and skips everything else. For analytical workloads, this is 10-100x faster and significantly smaller on disk.

Delta Lake is a storage layer on top of Parquet that adds three things: ACID transactions (if your pipeline fails mid-write, you get a clean rollback rather than a corrupt table), schema enforcement (columns cannot change types accidentally), and time travel (you can query what the table looked like three pipeline runs ago, which is useful for debugging).

This lakehouse pattern — raw data in Delta/Parquet, transformed models on top — is exactly what Databricks, Snowflake, and Azure Synapse use in production. Using it locally with the deltalake Python library (pure Python, no Spark required) demonstrates you understand the pattern without needing cloud resources.

The Delta tables live in data/parquet/, organized as data/parquet/raw/patients/, data/parquet/raw/encounters/, etc. These are your Bronze tier — raw, minimally processed, immutable.

What this phase produces: Delta tables in data/parquet/raw/ for each Synthea entity. An Airflow task that checks whether they exist before triggering regeneration (you don't want to regenerate and change your population on every pipeline run).

Phase 1B — Transformation Layer (dbt + DuckDB)

Goal: Transform raw Delta tables into clean, tested, documented analytical models.

Why DuckDB instead of SQLite:

DuckDB is an in-process analytical database that reads Parquet and Delta files natively — no loading or copying required. You point it at data/parquet/ and run SQL directly against the files. It is columnar internally, so analytical queries are dramatically faster than SQLite. The dbt-duckdb adapter connects dbt to DuckDB, meaning your dbt models read from Delta files and write results as DuckDB tables. This is a materially better architecture than loading CSVs into SQLite because your transformations operate directly on your lakehouse storage layer.

Why dbt:

dbt is how modern data teams write transformations. You write SQL SELECT statements; dbt handles the CREATE TABLE logic, runs models in dependency order, enforces a layered modeling pattern, runs tests, and generates documentation. The documentation is especially important here — the column descriptions you write in dbt become the schema context injected into the agent's SQL generation prompts. Better dbt documentation directly produces better SQL generation quality.

*Staging layer — stg_ models:**

One staging model per Synthea entity. These do minimal work: rename columns to consistent snake_case, cast data types, handle nulls explicitly, derive simple computed columns. No joins in staging.

stg_patients — renames all columns, casts BIRTHDATE and DEATHDATE to DATE, adds is_deceased (death_date IS NOT NULL), adds age_at_generation in years, adds age_bucket ('0-17', '18-44', '45-64', '65+').

stg_encounters — parses START and STOP as timestamps, computes duration_hours, keeps ENCOUNTERCLASS and REASONDESCRIPTION.

stg_conditions — handles nullable STOP explicitly (chronic conditions have no end date), keeps ICD-10 CODE and DESCRIPTION.

stg_observations — the most complex staging model. Splits the VALUE string column into value_numeric (cast to FLOAT where TYPE='numeric', else NULL) and value_text (VALUE where TYPE is not numeric). This split is what makes downstream lab queries clean.

stg_medications — adds is_active (STOP IS NULL), adds duration_days for completed medications.

*Mart layer — dim_ and fact_* models:**

dim_patients — starts from stg_patients, joins conditions to add chronic condition flags: has_diabetes (ICD-10 codes starting with E11), has_hypertension (I10), has_copd (J44), has_heart_failure (I50). Joins observations to add latest BMI and latest A1c. One row per patient, richly annotated.

fact_encounters — one row per encounter. Adds patient demographics, computes age at encounter, adds primary diagnosis description by joining conditions with overlapping dates. This is your most-queried table.

fact_lab_results — filtered from stg_observations where CATEGORY='laboratory'. Adds patient demographics. Adds is_abnormal flag using a dbt seed file mapping common LOINC codes to normal ranges — A1c above 5.7% is abnormal, fasting glucose above 100 mg/dL is abnormal, and so on. The fact that you know these clinical reference ranges signals domain awareness to interviewers.

fact_medications — one row per patient-medication pair. Adds drug_class from a seed file mapping RxNorm descriptions to classes (antihypertensive, statin, antidiabetic, anticoagulant). This seed is a CSV you write and commit to the dbt project.

Metrics layer — pre-aggregated views:

metrics_population_summary — total patients, alive count, deceased count, gender breakdown, average age. The agent hits this for overview questions rather than scanning dim_patients every time.

metrics_encounter_volume — encounter count by year-month and encounter class. For trend questions.

metrics_condition_prevalence — one row per condition with patient count and percentage of total population. For "what is the most common condition" questions.

dbt tests: Every primary key gets not_null and unique. Foreign keys get relationships tests. Categorical columns get accepted_values. Numeric columns get custom range tests. Run dbt test after every model run.

dbt documentation: Write a description for every model and every column in schema.yml. This is not busywork — it is what populates schema_registry.json, which is what the agent reads to generate SQL.

What this phase produces: Clean DuckDB tables materialized from Delta sources. A passing dbt test run. A generated schema_registry.json from dbt docs generate.

Phase 1C — Data Quality and Orchestration (Great Expectations + Airflow)

Goal: Validate data quality and wire the full pipeline into an Airflow DAG.

Great Expectations:

Great Expectations (GE) validates data by running typed assertions called Expectations against your tables. It produces an HTML Data Docs report showing pass/fail for every expectation. This is what data quality looks like in production — not ad hoc checks in Python scripts, but a documented, versioned suite that runs automatically.

The distinction from dbt tests: dbt tests catch structural problems (null keys, unexpected category values). GE catches domain-specific clinical quality issues. A hemoglobin A1c of 500% is structurally a valid number but clinically impossible. GE is where you encode clinical knowledge as validation rules.

Key expectation suites:

On dim_patients: age between 0 and 120. Gender in {M, F}. has_diabetes is 0 or 1 only. Income non-negative.

On fact_lab_results: A1c (LOINC 4548-4) between 3.0 and 15.0. Fasting glucose (LOINC 2339-0) between 20 and 600. Systolic BP (LOINC 8480-6) between 50 and 300. BMI (LOINC 39156-5) between 10 and 80.

On fact_encounters: duration_hours positive. start_datetime precedes end_datetime. total_cost non-negative.

GE runs after dbt in the Airflow DAG. If validation fails above a threshold, the downstream tasks do not run.

Airflow DAG:

The full batch pipeline DAG:

check_raw_data → run_dbt_staging → run_dbt_marts → run_dbt_tests → run_great_expectations → update_schema_registry

check_raw_data is a sensor task that checks whether the Delta tables exist. If they do, it skips regeneration. If they don't, it triggers the Synthea generation step. This matters because you don't want to regenerate your population (and change your benchmark answers) on every nightly run.

run_dbt_staging, run_dbt_marts are BashOperators. run_dbt_tests is a BashOperator that fails the DAG if any test fails. run_great_expectations is a PythonOperator.

update_schema_registry is a PythonOperator that parses dbt's manifest.json and writes schema_registry.json.

The DAG is scheduled nightly. In production this would trigger off new data arriving in the data lake. Here it is time-scheduled, which you note in your README.

Phase 2 — Semantic Layer (PostgreSQL + pgvector + RAG)

Goal: Add a vector search layer so the agent can answer questions from unstructured clinical documents.

Why RAG in a clinical context:

Not every clinical question is answerable from structured data. Questions about standards of care, clinical guidelines, readmission protocols, medication contraindications — these live in documents, not tables. A purely SQL-based agent cannot answer them.

RAG (Retrieval-Augmented Generation) retrieves relevant text from a document store and includes it in the LLM's prompt. The LLM uses the retrieved context to generate a grounded, cited answer rather than hallucinating from training knowledge.

pgvector is the PostgreSQL extension that adds vector similarity search. Using PostgreSQL means your structured patient data and your semantic document index live in the same database system. You can, in principle, join them — count the diabetic patients from the relational tables and retrieve the ADA guideline for diabetes from the vector table in the same query. That architectural coherence is worth noting.

The document corpus:

Fifteen to twenty clinical reference documents. Mix of publicly available clinical guidelines (ADA Standards of Care summary, ACC/AHA heart failure guideline key points, USPSTF preventive care recommendations, CMS readmission reduction program criteria) and institutional policy documents you write yourself (hospital readmission policy, medication reconciliation protocol, infection control procedure, escalation criteria). The self-written documents matter — they represent internal institutional knowledge that no public LLM knows, which makes RAG genuinely useful rather than cosmetic.

The embedding pipeline:

A Python script processes each document:

1. Chunks the text into overlapping segments — 400 tokens each with 50-token overlap. Overlapping prevents answers that span chunk boundaries from being missed.
2. Generates a 384-dimensional embedding vector for each chunk using sentence-transformers with the all-MiniLM-L6-v2 model. This model runs locally, is free, and is fast enough that embedding 20 documents takes seconds.
3. Stores each chunk in PostgreSQL: chunk_text, embedding (as a pgvector vector type), document_name, document_type (guideline, protocol, policy), chunk_index.

You create an IVFFlat index on the embedding column for approximate nearest neighbor search, which dramatically speeds up retrieval as the corpus grows.

Hybrid search — vector + keyword:

Pure vector search finds semantically similar text but can miss exact keyword matches. A clinician asking about "ICD-10 code E11.65" needs keyword matching, not semantic similarity. You implement hybrid search:

Vector search finds the top-10 chunks by cosine similarity to the query embedding. PostgreSQL full-text search (tsvector/tsquery) finds the top-10 chunks by keyword relevance. Reciprocal Rank Fusion merges and re-ranks both result sets — a standard, effective combination technique.

This is worth explaining in interviews. Most portfolio RAG implementations use only vector search. Hybrid search is what production RAG systems actually use, and knowing why is a differentiator.

How the agent uses RAG:

The intent classifier gets a document_query category. When the question is about guidelines, protocols, or standards of care, it routes to the RAG path.

The RAG node embeds the question, runs hybrid search, retrieves the top-5 chunks, formats them as context with source attribution, and passes them to the LLM: "Based on the following clinical guidelines, answer the question. Cite your sources." The response includes which document each claim came from.

The embedding pipeline in Airflow:

A DAG task update_vector_store runs whenever new documents are added to the documents folder. It checks which documents have already been embedded (by storing

document hashes in a tracking table) and only embeds new or changed documents. This is an incremental embedding pattern.

Phase 3 — The Agent Core (LangGraph + MCP + Ollama)

Goal: Wire everything from Phases 1-2 into a unified agentic reasoning system.

The agent state object:

```
class ClinIQState(TypedDict):
    user_question: str        # original question, never modified
    intent: str               # data_query / visualization / trend_analysis /
                              #   document_query / hybrid / out_of_scope
    schema_context: str       # relevant tables from schema_registry.json
    generated_sql: str        # from SQL generator
    rag_chunks: list          # retrieved document chunks from pgvector
    sql_result: list          # raw query result as list of dicts
    rag_result: str           # synthesized RAG answer
    sql_error: str            # last SQL execution error (for retry loop)
    retry_count: int          # SQL generation retry count, capped at 3
    validation_status: str    # valid / empty / out_of_range / suspect
    chart_type: str           # bar / line / scatter / pie / table / none
    chart_spec: dict          # Plotly figure as JSON-serializable dict
    final_explanation: str    # the plain English answer
    suggested_followup: str   # next question to ask
    mlflow_run_id: str
    tool_calls_log: list      # audit trail of every MCP tool call
    total_latency_ms: int
```

The nodes:

intent_classifier — sends the question to Ollama with a prompt defining all intent categories and asking for structured JSON output. Sets intent. Also sets a data_sources_needed field indicating whether the question needs SQL, RAG, or both.

schema_retriever — for SQL-bound intents, performs semantic search over your schema registry to find the most relevant tables. Rather than injecting the entire schema (which wastes context window space and confuses smaller models), you embed each table's description and retrieve only the top-5 most relevant tables for the question. This is how production Text-to-SQL handles large schemas and it's a meaningful upgrade over naive full-schema injection.

sql_generator — generates SQL with the relevant schema context, 3-5 few-shot examples in the prompt, and on retry the previous SQL plus the error message. Few-shot examples are crucial for local 7B models — they calibrate SQL style and prevent common mistakes. Validates that the output starts with SELECT before passing it forward.

sql_executor — executes SQL against DuckDB via the MCP database server (explained below). Enforces a 1000-row result limit. Captures specific error types: syntax errors are retryable, connection errors are fatal.

validator — runs rule-based checks (empty result, numeric values out of clinical ranges) followed by an LLM sanity check for ambiguous cases. The LLM sanity check sends the question, SQL, and first few result rows to Ollama and asks "does this result make sense as an answer to this question?" This catches semantic mismatches that rule-based checks miss — a query that executes successfully but answers the wrong question.

rag_retriever — embeds the question, runs hybrid search against pgvector via the MCP vector server, returns top-5 chunks with source attribution.

rag_synthesizer — takes the retrieved chunks and generates a cited answer using Ollama. Different prompt from the main response node — this explicitly asks the model to cite which document each claim comes from.

visualizer — selects chart type based on result shape (one categorical + one numeric → bar, one date + one numeric → line, two numeric → scatter, single number → metric card, more than 5 columns → table) and generates a Plotly figure. The figure is serialized to a JSON-compatible dict for the API response.

response_synthesizer — the final node. For SQL queries writes a 2-4 sentence explanation that directly answers the question, mentions the key number, notes any caveats (sample size, time period). For document queries uses the RAG synthesizer output. For hybrid

queries combines both. Always adds a suggested follow-up question. Always adds a caveat that this is synthetic data and should not be used for real clinical decisions.

The conditional edge logic:

After intent classification:

- data_query / visualization / trend_analysis → schema_retriever → sql_generator → sql_executor → validator → visualizer → response_synthesizer
- document_query → rag_retriever → rag_synthesizer → response_synthesizer
- hybrid → schema_retriever AND rag_retriever (parallel branches) → sql_generator → sql_executor → validator AND rag_synthesizer → visualizer → response_synthesizer (combines both)
- out_of_scope → response_synthesizer immediately with a helpful redirect message

After sql_executor failure:

- retry_count < 3 → sql_generator (with error in prompt)
- retry_count >= 3 → response_synthesizer (with failure message)

After validation:

- valid → visualizer
- empty → response_synthesizer (with "no matching data found" message)
- suspect → sql_generator (with validation failure context in prompt)

MCP — why and how:

Rather than having agent nodes call DuckDB and PostgreSQL directly, they call through MCP servers. MCP (Model Context Protocol) is Anthropic's open protocol for standardizing how agents connect to data sources and tools. You build two MCP servers:

mcp_servers/database_server.py exposes three tools: query_clinical_database (takes SQL, validates it is SELECT-only, executes against DuckDB, returns results as JSON), get_schema_info (returns fresh schema context for specified tables), get_metric_summary (returns precomputed metric values from the metrics dbt layer).

mcp_servers/vector_server.py exposes two tools: search_clinical_documents (takes a query string, runs hybrid search, returns top-k chunks with sources) and list_document_sources (returns the list of available documents — useful for the frontend to show users what knowledge base is available).

Why this matters: the agent never has direct database credentials. Every data access goes through an MCP server that enforces read-only access, logs every call, and enforces row limits. This is the security boundary pattern that enterprise AI deployments require. It also means you can swap the underlying database without changing agent code — replace DuckDB with Databricks SQL by writing a new MCP server that speaks Databricks. The agent doesn't know or care.

Ollama — local LLM inference:

Ollama runs as a Docker service. You pull three models: mistral:7b (default, fast, good structured output), llama3.1:8b (used for the response synthesizer where quality matters most), codellama:7b (optionally used for the SQL generator where code generation quality matters). Which model each node uses is configured in config.py.

The multi-model-per-node architecture is a genuine talking point. Most portfolio projects use one model for everything. Treating model selection as a per-node decision demonstrates that you understand the cost-quality tradeoff differently for classification (fast is fine), SQL generation (code-specialized is better), and natural language synthesis (quality matters most).

Phase 4 — Experiment Tracking and Evaluation (MLflow)

Goal: Track every agent run systematically and build a quantitative benchmark that produces numbers for your resume.

MLflow setup:

MLflow tracking server runs as a Docker service. Every agent invocation starts an MLflow run, logs all relevant information, and ends the run with the response. This gives you a complete trace for every question ever asked to the system.

What you log per run:

Parameters: model names per node, prompt version per node (prompts live in prompts.py with version comments), retry limit, hybrid search weights, embedding model name.

Metrics: SQL execution success (1/0), answer accuracy (1/0, from benchmark comparison), retry count, total latency in milliseconds, RAG chunk count, validation status as a numeric code.

Artifacts: the generated SQL query, the retrieved RAG chunks as JSON, the Plotly chart spec, the final explanation text.

The evaluation benchmark:

Twenty-five benchmark questions with verified correct answers. The correct answers are determined by you running SQL manually against DuckDB with the seeded Synthea population before building the agent.

SQL questions (15): "How many patients are in the dataset?", "What percentage of patients are female?", "How many patients have Type 2 diabetes?", "What is the most common encounter class?", "What is the average inpatient stay duration in days?", "Which month has the highest ER visit volume?", "How many patients are currently on metformin?", "What is the average A1c for diabetic patients?", "What percentage of encounters are emergency visits?", "How many patients have both diabetes and hypertension?", and five more.

RAG questions (10): "What are the ADA's A1c targets for elderly diabetic patients?", "What does the readmission policy say about heart failure patients?", "Which medications are noted as contraindicated in patients with kidney disease in your protocols?", "What are the escalation criteria for abnormal lab results?", and six more.

The evaluation script runs all 25 questions through the full agent, compares answers to known correct values, and logs pass/fail to MLflow. Expected result for a well-tuned agent: 80-85% SQL accuracy, 75-80% RAG accuracy. Those numbers go in your README, your resume, and your interview answers.

Prompt versioning with MLflow: Every time you change a prompt in prompts.py, increment its version comment. When you run the benchmark, the prompt version is logged as a parameter. This lets you track "prompt v3 for SQL generation achieved 83% vs prompt v2 at 74%" in the MLflow experiment comparison view.

Phase 5 — API, Frontend, and Deployment

Goal: Wrap the agent in a production-grade API and build a compelling demo UI.

FastAPI backend:

POST /ask — primary endpoint. Request: {"question": str, "mode": "auto" | "sql" | "rag"}. Response: {"answer": str, "sql": str, "chart_spec": dict, "rag_sources": list, "intent": str, "run_id": str, "latency_ms": int}.

GET /health — returns service health including DuckDB connection status, PostgreSQL connection status, and Ollama model availability.

GET /schema — returns the schema registry, used by the frontend to show users what tables are available and what questions are answerable.

GET /examples — returns example questions by intent category, used by the frontend's example panel.

GET /run/{run_id} — returns the full MLflow trace for a past run. Used by the history panel.

Streamlit frontend:

Three panels:

Query interface — text input, submit button, an intent badge (colored label showing what the agent classified the question as), the explanation text, the interactive Plotly chart, and the suggested follow-up question.

Transparency panel — expandable section showing the generated SQL, the RAG sources cited with document names and chunk excerpts. Showing the SQL is a design principle, not a concession — clinical users need to be able to audit answers. It also shows every technical reviewer that the SQL generation is actually working.

History panel — recent queries with their intent classification, whether they passed validation, and a link to the MLflow trace. Demonstrates that experiment tracking is real and working, not just claimed.

HuggingFace Spaces deployment:

The deployed version swaps Ollama for the Groq free API (Llama 3.1 8B, extremely fast, no credit card required for basic use) via an environment variable. PostgreSQL + pgvector swaps for Neon (free tier, pgvector supported). DuckDB runs in-process with Parquet files committed to the Spaces repo. All of this is noted honestly in the README.

Docker Compose:

docker compose up starts the full local stack. Health checks on each service ensure dependent services wait for their dependencies. A Makefile with a run target handles the startup sequence and a test target runs the benchmark evaluation.

Technology Stack — Full Summary

Category	Tool	Why
Synthetic data	Synthea	ICD-10/LOINC/RxNorm coded, free, local, realistic longitudinal histories
Storage format	Parquet + Delta Lake	Columnar efficiency, ACID writes, time travel, lakehouse pattern
Query engine	DuckDB	Reads Parquet/Delta natively, columnar, zero-config, dbt adapter available
Transformation	dbt (dbt-duckdb)	Industry standard, tested, documented, lineage, column descriptions feed agent
Data quality	Great Expectations	Clinical-domain validation, HTML reports, production-grade pattern
Orchestration	Airflow	Most recognized orchestrator in job postings, DAG visibility
Vector store	PostgreSQL + pgvector	Hybrid SQL + semantic search in one system, no separate vector DB to manage
Embeddings	sentence-transformers	Free, local, 384-dim, production-capable, no API cost
Agent framework	LangGraph	Explicit stateful graph, conditional edges, inspectable, debuggable
LLM inference	Ollama (Mistral 7B, LLaMA 3.1 8B, CodeLLaMA 7B)	Local, free, no data leaves machine, multi-model-per-node pattern
Tool protocol	MCP	Anthropic standard, security boundary pattern, swap backends without changing agent
Experiment tracking	MLflow	Standard for AI/ML run tracking, benchmark quantification
API	FastAPI	Python ML service standard, async, OpenAPI docs auto-generated

Category	Tool	Why
Frontend	Streamlit	Data-focused, Plotly native, fastest path to working demo
Visualization	Plotly	Interactive, renders in Streamlit, serializes to JSON for API
Containerization	Docker + Docker Compose	One-command startup, engineering professionalism signal
Deployment	HuggingFace Spaces	Free, Streamlit native, widely recognized

---

The Interview Narrative

"Clinical data teams have an analyst bottleneck. Every non-technical stakeholder — a nurse manager, a quality officer, a finance director — is blocked waiting for a data analyst to write a query. ClinIQ eliminates that wait.

It's a locally deployable clinical intelligence platform. Non-technical users ask questions in plain English. The agent classifies the intent and routes to the right retrieval path — SQL for structured data questions, RAG over a pgvector document store for guideline and protocol questions. When SQL fails, the agent corrects itself using the error message as context and retries.

The data layer is a lakehouse architecture — Synthea synthetic EHR data lands as Delta tables in Parquet format, transformed by dbt models on DuckDB, orchestrated by Airflow. Great Expectations runs clinical-domain validation rules after every pipeline run — things like catching an A1c value of 500%, which is structurally valid but clinically impossible.

Every agent run is tracked in MLflow. I ran a 25-question benchmark across both SQL and RAG questions. The agent answers 83% of SQL questions correctly and 78% of RAG questions. The SQL failures are mostly date-window queries where the model misinterprets relative time without knowing today's date — a solvable prompt engineering problem.

In production I'd replace DuckDB with Databricks SQL and Ollama with a fine-tuned Text-to-SQL model. But the architecture is the same. The agent doesn't know what's behind the MCP servers — swapping backends is a configuration change, not a code change."